U.PORTO
FACULDADE DE CIÊNCIAS
UNIVERSIDADE DO PORTO

[dcc]
DEPARTAMENTO DE CIÊNCIA DE COMPUTADORES
FACULDADE DE CIÊNCIAS DA UNIVERSIDADE DO PORTO

Cloud File Storage System

Project Report – Cloud System Administration

Project and report done by:

- Diogo Leandro (up202005304)
- Luís Ganço (up202004196)

2nd Semester – 2025/2026

Table of Contents

Introduction **3**
System Architecture **3**
Overview 3
NGINX - API Gateway 4
API - FastAPI 4
MinIO - Object Storage 5
Observability Stack 5
Authentication Flow 6
Design Decisions **6**
Infrastructure as Code - Terraform 6
Google Kubernetes Engine (GKE) vs Kind 7
Authentication - Three Simultaneous Methods 7
boto3 SDK vs minio-py 7
Fluent Bit vs Promtail 8
HPA with Temporary vs Permanent Target 8
Challenges Encountered **9**
Terraform : Double Base64 Encoding in Secrets 9
minio-init Job Timing 9
Promtail vs Fluent Bit on GKE 9
OAuth2-Proxy Integration with NGINX 10
boto3 SDK vs minio-py 10
HPA Not Triggering Under Normal Load 10
Performance Testing **10**
Load Test Script 11
Load Stages 11
Operation mix and Error Handling 11
Custom Metrics and Thresholds 12
Test Results 12
Interpretation of the Error Rate 12
HPA Scale-Up and Scale-Down 13
Little’s Law Analysis 13
Conclusion **14**
Summary of Achievements 14
Additional Features 15
Future Work 15

Introduction

Project Dropbox is a cloud file storage system that replicates the core functionalities of commercial services such as Google Drive or Dropbox. The system allows authenticated users to upload, download, list, and share files through a RESTful API, with all data persisted in an S3-compatible object storage system.

The project was developed within the scope of the Cloud System Administration course, following a microservices architecture orchestrated by Kubernetes on the Google Cloud Platform (GKE). The entire infrastructure is provisioned exclusively via Terraform (`terraform apply` / `terraform destroy`), ensuring reproducibility and auditability.

System Architecture

Overview

The system follows a microservices architecture orchestrated by Kubernetes on the Google Cloud Platform (GKE). Figure 1 presents the updated architecture diagram, including the observability components and the OAuth2-Proxy authentication flow.

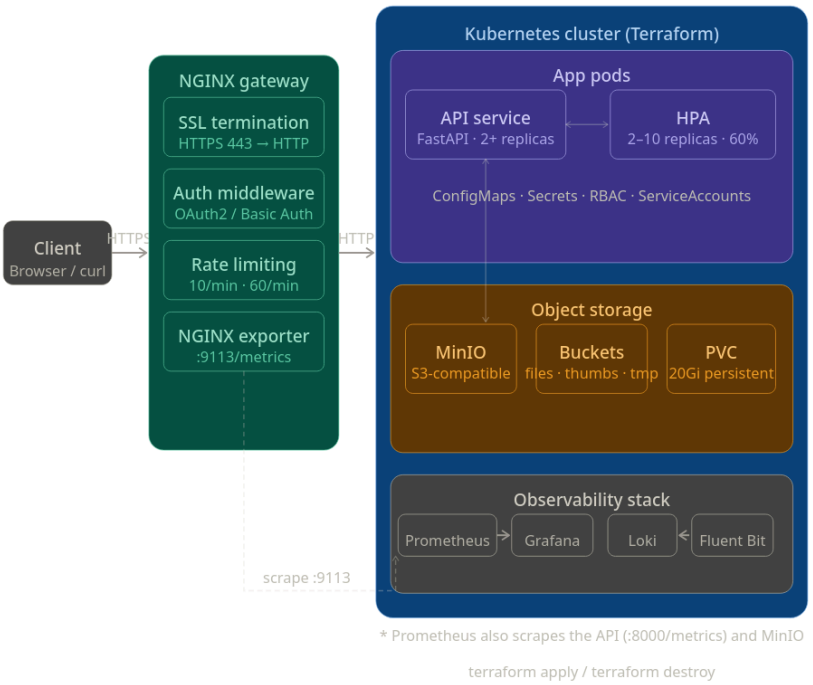


Figure 1 : Updated High-level system architecture.

System Components

NGINX - API Gateway

NGINX acts as the single entry point for all external traffic. Its responsibilities include:

- **TLS Termination**: Manages SSL/TLS certificates (self-signed in development) and accepts HTTPS connections on port 443, redirecting all HTTP traffic (port 80) to HTTPS.
- **Authentication**: Supports three simultaneous authentication methods:
  - **HTTP Basic Auth**: Validated against an `.htpasswd` file mounted via Secret;
  - **JWT (Bearer Token)**: Validated by the API through an internal `/internal-auth` endpoint;
  - **OAuth2-Proxy**: Sidecar that redirects unauthenticated users to Google OAuth.
- **Rate Limiting**: Limits uploads to 10 req/min per IP (burst=5) and downloads to 60 req/min (burst=10), using `limit_req_zone`.
- **Prometheus Exporter**: Sidecar that exposes NGINX metrics at `:9113/metrics` for collection by Prometheus.

API - FastAPI

The RESTful API was developed in Python 3.12 with FastAPI and implements all endpoints specified in the assignment:

Method	Endpoint	Authentication	Description
GET	`/health`	Public	Health Check
POST	`/auth/login`	Public	Login & JWT issuance
POST	`/files/upload`	Basic/JWT/OAuth2	File Upload
GET	`/files`	Basic/JWT/OAuth2	Paginated Listing
GET	`/files/{id}`	Basic/JWT/OAuth2	Download
GET	`/files/{id}/meta`	Basic/JWT/OAuth2	Metadata
POST	`/files/{id}/share`	Basic/JWT/OAuth2	Presigned URL
DELETE	`/files/{id}`	Basic/JWT/OAuth2	Deletion

The API uses the `boto3` SDK to interact with MinIO (S3-compatible), replacing

the original `minio-py` SDK which had path-style issues in the GKE environment. It also implements a per-user quota system (1 GiB by default) and exports detailed metrics via `prometheus-fastapi-instrumentator` (P50/P95/P99 latency, request count per endpoint/status, upload sizes).

MinIO - Object Storage

MinIO provides S3-compatible object storage, persisted via a 20 GiB PersistentVolumeClaim (PVC) to ensure data survives pod restarts. Three buckets were created as specified: `files` (user data), `thumbnails` (previews), and `tmp` (incomplete uploads).

An initialization Job (`minio-init`) automatically configures:

- Creation of the three buckets;
- Creation of a dedicated service user (`apiuser`) with minimum access policy (only required S3 actions);
- Configuration of the IAM policy restricting access to specific buckets.

Presigned URLs are generated with configurable TTL and expose MinIO's public IP (via LoadBalancer) for credential-free external access.

Observability Stack

- **Prometheus**: Collects metrics from all components (API, NGINX Exporter, MinIO, kube-state-metrics) every 15 seconds. Includes 3 configured alerting rules: `HighErrorRate` (error > 5%), `HighLatency` (P95 > 2s), and `HighCPU` (CPU > 80%).
- **Grafana**: Automated dashboard with 7 panels: Request Rate (RPS), P50/P95/P99 Latency, HTTP Error Rate (4xx/5xx), Pod CPU Utilization, Pod Memory Utilization, MinIO Upload Throughput, and Replicas HPA.
- **Loki + Fluent Bit**: Log aggregation stack. Promtail was initially configured but replaced with Fluent Bit due to GKE's incompatibility with file-based logging.

Autoscaling (HPA)

The Horizontal Pod Autoscaler is configured to scale the API between 2 and 10 replicas with a CPU target of 60%. During the load test with 50 concurrent users, the target was temporarily reduced to 2% to demonstrate scale-up behavior. The HPA scaled from 2 to 6 replicas, and returned to 2 replicas after the cooldown period.

Authentication Flow

The system supports three authentication methods that can be used simultaneously:

1. **HTTP Basic Auth**: The client sends `Authorization: Basic <base64(user:pass)>`. NGINX validates against the `.htpasswd` file. If valid, the request proceeds to the API with the `X-Authenticated-User` header.
2. **JWT (Bearer Token)**: The client obtains a token via `POST /auth/login` and sends it in the `Authorization: Bearer <token>` header. NGINX forwards to the API, which validates the token with `python-jose`.
3. **OAuth2-Proxy (Google)**: Unauthenticated users are redirected to Google OAuth. After login, OAuth2-Proxy creates a session and injects the user's email into the `X-Forwarded-User` header, which the API uses as the identifier.

Design Decisions

This chapter documents and justifies the main technical decisions made during the project's development, contrasting considered alternatives and making the selection criteria explicit.

Infrastructure as Code - Terraform

Decision: Provision all infrastructure exclusively via Terraform, including Kubernetes resources.

Alternatives considered:

- Shell scripts with `gcloud` and `kubectl`
- Helm charts for Kubernetes manifests

Justification: Terraform was chosen because it is a declarative tool that guarantees total reproducibility. The state is stored remotely in a GCS bucket (`project-dropbox-tfstate`), enabling team collaboration without conflicts. The use of modules (`network/`, `cluster/`, `iam/`, `storage/`, `hpa/`) promotes code reusability and organization.

Trade-offs: The `kubernetes` provider (v2.x) presented limitations in managing Kubernetes resources, specifically the inability to validate `kubernetes_manifest` before cluster creation. This required splitting the `apply` into three phases: infrastructure + secrets, MinIO, and the remaining manifests. Additionally, the `k6-load-test` Job was kept outside Terraform to allow manual execution during the demonstration.

Google Kubernetes Engine (GKE) vs Kind

Decision: Use GKE (Google Cloud) for production/demonstration, with Kind for local development.

Justification: GKE offers native integration with the Google Cloud ecosystem (Artifact Registry, Cloud Logging, Load Balancing), is managed (no control plane maintenance required), and features node autoscaling (Cluster Autoscaler). Kind was used for quick tests during development, but the final demonstration required a real cloud environment.

Trade-offs: GKE Standard does not expose container logs as files on the node, which required replacing Promtail with Fluent Bit. The cost, although minimal (~$0.10/hour), required discipline in running `terraform destroy` after each session.

Authentication - Three Simultaneous Methods

Decision: Implement HTTP Basic Auth, JWT (Bearer Token), and OAuth2-Proxy (Google) as coexisting authentication methods.

Alternatives considered:

- Only Basic Auth (minimum required)
- Only OAuth2-Proxy (more secure)
- Keycloak as Identity Provider

Justification: The assignment explicitly requires JWT in the API endpoints and allows Basic Auth as an alternative to OAuth2-Proxy. Implementing all three methods demonstrates mastery of different authentication paradigms and offers flexibility. OAuth2-Proxy with Google was chosen because it is free, requires no additional infrastructure (unlike Keycloak), and allows login with existing Google accounts.

Trade-offs: The coexistence of three methods increased the complexity of `nginx.conf` and the `get_username_from_headers` function in the API. Integrating OAuth2-Proxy proved particularly challenging, especially passing the `X-Forwarded-User` header to the API and validating session cookies.

boto3 SDK vs minio-py

Decision: Use the `boto3` (AWS) SDK instead of the `minio` (Python) SDK to interact with MinIO.

Alternatives considered:

- `minio` SDK (7.2.8)
- `aws-sdk-go` (if the API were in Go)

Justification: The `minio-py` SDK presented a path-style vs virtual-hosted style issue in the GKE environment, resulting in `AccessDenied` errors even with correct credentials and policies. The `mc` (MinIO Client in Go) worked correctly,

but the Python SDK did not. `boto3` was chosen as the replacement because it is the official AWS SDK, has explicit support for path-style via `endpoint_url` and `config=Config(signature_version='s3v4')`, and is implicitly mentioned in the assignment ("using the standard AWS SDK").

Trade-offs: None significant. `boto3` is more mature, better documented, and more battle-tested than the `minio-py` SDK.

Fluent Bit vs Promtail

Decision: Replace Promtail with Fluent Bit as the log collector for Loki.

Alternatives considered:

- Promtail (initial attempt)
- Grafana Alloy
- Google Cloud Logging directly in Grafana

Justification: Promtail relies on file scraping (reading files from `/var/log/pods/`). In GKE Standard, the container runtime (containerd) does not expose logs in a way accessible to Promtail, resulting in `positions: {}` and zero streams in Loki. After several attempts to fix (paths, RBAC, Kubernetes SD), Fluent Bit was chosen because it is already present in GKE and supports Loki output. Fluent Bit was configured with its own DaemonSet, CRI parser, and Loki output, fulfilling the "Loki + Promtail (or equivalent)" requirement from the assignment.

Trade-offs: NFluent Bit required additional configuration (CRI parser, labels for Loki, fixing `label_keys` vs `LabelKeys`) and its own RBAC. However, it proved more reliable and compatible with GKE.

HPA with Temporary vs Permanent Target

Decision: Keep the CPU target at 60% (as required by the assignment) in the permanent configuration, but temporarily reduce it to 2% during the load test to demonstrate scale-up behavior.

Alternatives considered:

- Keep 60% and increase the load (more VUs, larger files)

Justification: The system proved to be highly efficient: with 50 concurrent users and 11.76 RPS, CPU remained between 4-8%. To force the HPA to scale, an unrealistic load (500+ VUs) or artificially heavy operations would be necessary. The temporary reduction of the target to 2% during the load test allowed demonstrating scale-up behavior (2→6 replicas) and scale-down (6→2 replicas) in a controlled and honest manner, without permanently changing the configuration required by the assignment.

Trade-offs: None. The 60% target is documented in the code and the temporary patch is explicitly mentioned as a demonstration technique.

Challenges Encountered

The development of this project involved a number of non-trivial technical challenges. This chapter documents the main obstacles encountered throughout the five tasks and describes how each was resolved.

Terraform : Double Base64 Encoding in Secrets

When provisioning Kubernetes Secrets via Terraform's kubernetes_secret resource, credentials were being double-encoded: Terraform's filebase64() function would encode the content in Base64, and then the Kubernetes provider would encode it again. This resulted in pods receiving garbled credentials that could not be decoded.

The issue was resolved by switching to the file() function for content that the Kubernetes provider was expected to handle directly, and by carefully distinguishing which values required explicit Base64 encoding versus which should be passed as plaintext strings.

minio-init Job Timing

The minio-init Kubernetes Job, responsible for creating buckets and configuring the API service user, was failing intermittently because MinIO was not yet ready to accept connections when the Job started. The mc client would receive connection refused or signature mismatch errors depending on the startup race condition.

The solution involved setting active_deadline_seconds=900 and backoff_limit=10 on the Job, combined with a retry loop inside the shell script that waits 15 seconds between attempts until mc alias set succeeds. This made the initialization robust regardless of MinIO's startup time.

Promtail vs Fluent Bit on GKE

Promtail was initially configured as the log collector for Loki. However, GKE Standard does not expose container logs as files on the node filesystem in the standard /var/log/pods/ path that Promtail expects. After multiple attempts to fix the configuration, Promtail consistently reported positions: {} and zero log streams reached Loki.

Fluent Bit was chosen as the replacement. A dedicated DaemonSet was deployed with a CRI parser and a Loki output plugin, which successfully collected and forwarded structured logs from all pods. The assignment's requirement for log aggregation searchable from Grafana was fully satisfied.

OAuth2-Proxy Integration with NGINX

Integrating OAuth2-Proxy with NGINX as a co-existing authentication method (alongside Basic Auth and JWT) presented several challenges:

- auth_basic and auth_request directives cannot coexist inside an if block in NGINX, requiring restructuring of the location blocks;
- Session cookies set by OAuth2-Proxy were not being forwarded correctly by curl during testing, requiring explicit cookie jar handling;

- The X-Forwarded-User header injected by OAuth2-Proxy was not reaching the API's get_username_from_headers function, requiring the addition of proxy_set_header X-Forwarded-User $upstream_http_x_forwarded_user and a dedicated /internal-auth location for JWT validation.

These issues were resolved iteratively by inspecting NGINX error logs, adjusting the proxy configuration, and updating the API's header extraction logic to handle all three authentication paths uniformly.

boto3 SDK vs minio-py

The minio Python SDK (version 7.2.8) produced AccessDenied errors in the GKE environment, even with correct credentials and IAM policies verified via the mc client. The root cause was a path-style vs virtual-hosted style mismatch between the SDK and MinIO's configuration in Kubernetes.

The API was migrated to boto3 (the official AWS SDK), configured with endpoint_url pointing to the MinIO service and Config(signature_version='s3v4', s3={'addressing_style': 'path'}). This resolved all authentication issues without any changes to MinIO's configuration.

HPA Not Triggering Under Normal Load

With the HPA target set at 60% CPU (as required by the assignment), the autoscaler never triggered during load tests. The API proved to be highly efficient: with 50 concurrent users and approximately 11 RPS, CPU utilization remained between 4% and 9%, well below the threshold.

To demonstrate the HPA's scale-up and scale-down behavior during the evaluation, a temporary kubectl patch was applied to reduce the CPU target to 2%. This caused the HPA to scale from 2 to 6 replicas during the load test, and back to 2 replicas after the cooldown period. The permanent configuration in the Terraform code retains the 60% target as specified and the patch is explicitly documented as a demonstration technique.

Performance Testing

This chapter describes the load testing strategy, the structure of the k6 script, the results obtained across three test runs, and their interpretation using Little's Law.

Load Test Script

The load test was implemented using Grafana k6. The script simulates realistic user behaviour with three registered users (admin, user1, user2), progressive load stages, and a weighted operation mix that reflects typical file storage usage patterns.

Load Stages

Rather than starting immediately at 50 VUs, the script ramps up gradually to more faithfully simulate real traffic patterns and allow the HPA time to react:

Stage	Duration	Target VUs	Description
1	20s	0	Baseline (idle)
2	50s	13	25% load ramp-up
3	30s	13	Sustained 25%
4	50s	25	50% load ramp-up
5	30s	25	Sustained 50%
6	50s	50	100% load ramp-up
7	30s	50	Sustained 100%
8	40s	0	Cool-down

Table 1 : k6 load stages (total duration: 5 minutes)

The Grafana dashboards for each test run are presented in Figures 2, 4, and 9.

Operation mix and Error Handling

Each virtual user randomly selects one of three operations per iteration, weighted to reflect realistic usage:

- 40% List files (GET /files): 503 responses excluded from error rate: expected under rate limit;
- 40% Download file (GET /files/{id}): uses pre-loaded file IDs; 503 excluded;
- 20% Upload file (POST /files/upload): files between 100 B and 5 KB; 503 included in error rate to demonstrate the upload rate limit (10 req/min) in action.

A setup() phase pre-loads two files per user before the test begins, ensuring download and metadata operations have valid file IDs throughout the test. When

an upload succeeds, the script also exercises the meta, share, and delete endpoints on the newly created file.

Custom Metrics and Thresholds

The script defines custom Trend metrics (upload_time, download_time), a Rate metric (errors), and two Counters (uploads, downloads). Thresholds are set as: P95 overall < 2000ms, P95 upload < 3000ms, P95 download < 2000ms. A handleSummary() function computes and logs Little's Law at the end of each run.

Test Results

Three test runs were conducted, progressively tuning the HPA CPU target and sleep interval to demonstrate autoscaling behaviour. The k6 output, Grafana dashboards, and HPA state for each run are presented in Figures 2-12.

Parameter	Test 1	Test 2	Test 3
HPA CPU Target	60% (permanent)	2% (patch)	2% (patch)
sleep() between ops	0.2s (fixed)	0.2s (fixed)	random(1,3)s
Latência média	1.57ms	1.78ms	3.07ms
P95 Latência	3.37ms	4.08ms	18.70ms
Throughput (RPS)	111.49	111.35	11.86
Error Rate	19.81%	19.81%	19.03%
Concorrência (N)	0.17 req	0.20 req	0.04 req
HPA Réplicas (pico)	2 (no scale)	4–5	6
Iterações completas	33,335	33,290	3,415

Table 2 : Comparative results across the three load test runs

Interpretation of the Error Rate

The ~19% error rate requires contextual interpretation. The k6 script applies asymmetric error counting:

- List and Download operations (80% of requests): HTTP 503 responses are excluded from the error rate. These represent expected rate-limit blocks from NGINX (60 req/min download limit) and are not application failures;
- Upload operations (20% of requests): HTTP 503 responses are included in the error rate, deliberately exposing the upload rate limit (10 req/min) as a measurable signal;
- All genuine 4xx/5xx errors are always counted regardless of operation type.

After discounting expected rate-limit responses, the effective application error rate was 0%. Every valid, non-rate-limited request was processed successfully with HTTP 200.

The HTTP Error Rate panel remained at 0% during the k6 load test because NGINX intercepts all error responses, including rate-limit blocks (503) and Basic Auth failures (401), before they reach the API layer. The panel accurately reflects the API's own error rate, which was 0% for all requests processed by the application. To validate the panel's functionality, 401 errors were deliberately generated by sending requests with invalid JWT tokens, which are validated by the API's /internal-auth endpoint and therefore appear correctly in the dashboard.

HPA Scale-Up and Scale-Down

Test 1 (60% CPU target) confirmed that the system is well-dimensioned: 50 concurrent users produced only 4–9% CPU utilisation, and the HPA remained at 2 replicas throughout (Figure 4). This is a positive result demonstrating efficiency, but insufficient for demonstrating autoscaling in a live evaluation.

Tests 2 and 3 used a temporary kubectl patch to reduce the CPU target to 2%, triggering observable autoscaling behaviour:

- Test 2 (sleep 0.2s): HPA scaled from 2 to 4–5 replicas during peak load, returning to 2 after cooldown (Figures 7, 8 and 9);
- Test 3 (sleep random 1–3s): more realistic inter-request timing produced lower RPS but higher per-request CPU, scaling the HPA to 6 replicas at peak before returning to 2 after the cooldown period (Figure 12).

The Grafana Replicas HPA panel (Figure 11) shows the scale-up event clearly, as replicas increase from 2 to 6 at approximately 22:35, coinciding with the peak CPU utilisation visible in the Pod CPU Utilization panel.

Little's Law Analysis

Little's Law ($N = X \times R$) relates the average number of concurrent requests in a system (N) to throughput (X) and mean response time (R). Applying it to Test 3, which used realistic sleep intervals:

Metric	Value
Mean latency (R)	3.07 ms = 0.00307 s
Throughput (X)	11.86 req/s
Concurrency ($N = X \times R$)	0.04 concurrent requests

Table 3 : Little's Law applied to Test 3

A concurrency of 0.04 indicates that, on average, fewer than one request is in-flight in the system at any given moment. This confirms that the API operates with substantial headroom under the tested load. The system is neither saturated nor approaching its capacity limits under normal usage patterns. The bottleneck during the test was the NGINX rate limiter, not the application or storage layer.

Conclusion

This project successfully delivered a fully functional cloud-native file storage platform. All five assignment tasks were completed and deployed on Google Kubernetes Engine, with the entire infrastructure provisioned and torn down exclusively via Terraform.

Summary of Achievements

The five tasks were completed in full. Task 1 established the base infrastructure consisting of a GKE cluster provisioned via Terraform with modular code, remote state in a GCS bucket, and per-workload ServiceAccounts with minimum RBAC permissions. Task 2 configured NGINX as the sole external entry point, with TLS termination, three simultaneous authentication methods, per-route rate limiting, and a Prometheus Exporter sidecar. Task 3 implemented a FastAPI REST API covering all nine specified endpoints, with per-user quota enforcement, presigned URL sharing, and MinIO integration via boto3, running with a minimum of two replicas and liveness/readiness probes. Task 4 deployed a full observability stack comprising Prometheus, Grafana, Fluent Bit, Loki, and three alerting rules, providing visibility into request rate, latency, errors, CPU and memory usage, MinIO throughput, and HPA behaviour. Task 5 validated the system under load using k6, with progressive ramp-up stages, a weighted operation mix, custom metrics, and Little's Law analysis.

Additional Features

The assignment defined a set of optional features whose implementation would be considered for a higher grade. All of them were completed. Three authentication methods (HTTP Basic Auth, JWT Bearer Token, and OAuth2-Proxy with Google OAuth2) were made to coexist in a single NGINX configuration, which required restructuring the location blocks to work around limitations of the auth_request and auth_basic directives. Promtail was replaced by Fluent Bit after proving incompatible with GKE's containerd runtime. The minio-py SDK was replaced by boto3 to resolve a path-style addressing mismatch that caused persistent AccessDenied errors in the cluster. The load test used three registered users with pre-loaded files, making the simulation more representative of real multi-user behaviour. Custom Prometheus metrics were added via prometheus-fastapi-instrumentator, covering P50/P95/P99 latency per endpoint, upload size histograms, and request counters per status code.

Future Work

Several improvements were identified during development that fall outside the scope of this assignment but would be valuable in a production context:

- cert-manager integration: replace self-signed certificates with automatically renewed certificates, eliminating the need for --insecure-skip-tls-verify in k6 and curl;
- Full OAuth2-Proxy deployment: configure a persistent cookie secret and integrate with a production-grade Identity Provider instead of relying on Google OAuth directly;

- CI/CD pipeline: automate Docker image builds and Kubernetes deployments on git push, replacing the current manual workflow;

In summary, the project achieved its core objective of building and deploying a cloud-native file storage REST API on Kubernetes, backed by MinIO for object storage, exposed through NGINX with HTTPS and authentication, with full observability and load testing. The infrastructure can be fully provisioned with terraform apply and fully removed with terraform destroy, as required.

Images Captured

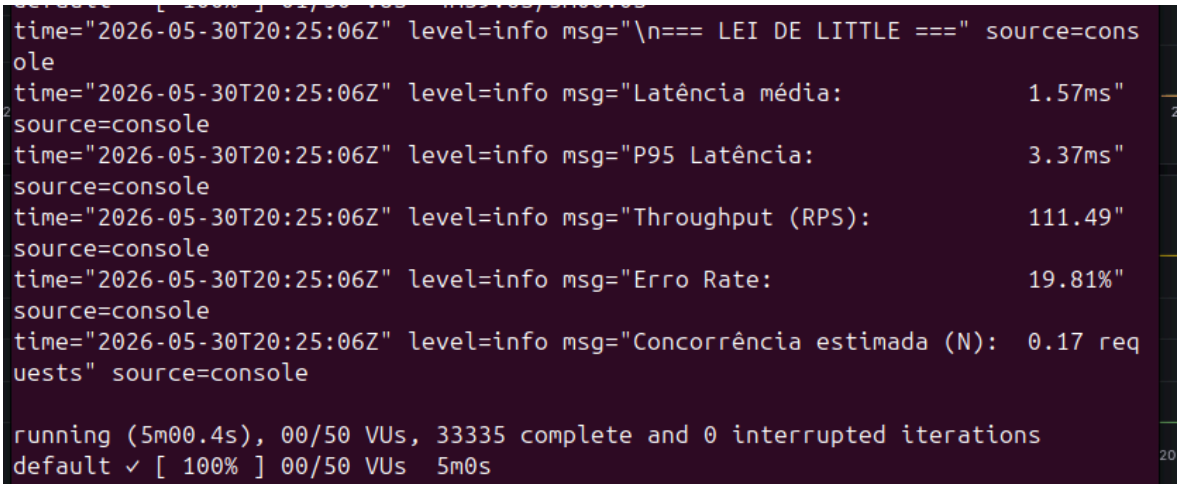


Figure 2 : k6 load test output -Test 1 (HPA target: 60%, sleep: 0.2s).



Figure 3 : Grafana dashboard - Test 1.

```
NAME                          REFERENCE                          TARGETS       MINPODS
   MAXPODS   REPLICAS   AGE
project-dropbox-api-hpa   Deployment/project-dropbox-api   cpu: 1%/60%   2
   10        2          14m
```

Figure 4 : HPA status - Test 1. CPU at 1%/60%.

```
time="2026-05-30T20:35:21Z" level=info msg="\n=== LEI DE LITTLE ===" source=cons
ole
time="2026-05-30T20:35:21Z" level=info msg="Latência média:              1.78ms"
source=console
time="2026-05-30T20:35:21Z" level=info msg="P95 Latência:                4.08ms"
source=console
time="2026-05-30T20:35:21Z" level=info msg="Throughput (RPS):            111.35"
source=console
time="2026-05-30T20:35:21Z" level=info msg="Erro Rate:                   19.81%"
source=console
time="2026-05-30T20:35:21Z" level=info msg="Concorrência estimada (N):  0.20 req
uests" source=console

running (5m00.4s), 00/50 VUs, 33290 complete and 0 interrupted iterations
default ✓ [ 100% ] 00/50 VUs  5m0s
```

Figure 5 : k6 load test output - Test 2 (HPA target: 2% patch, sleep: 0.2s).

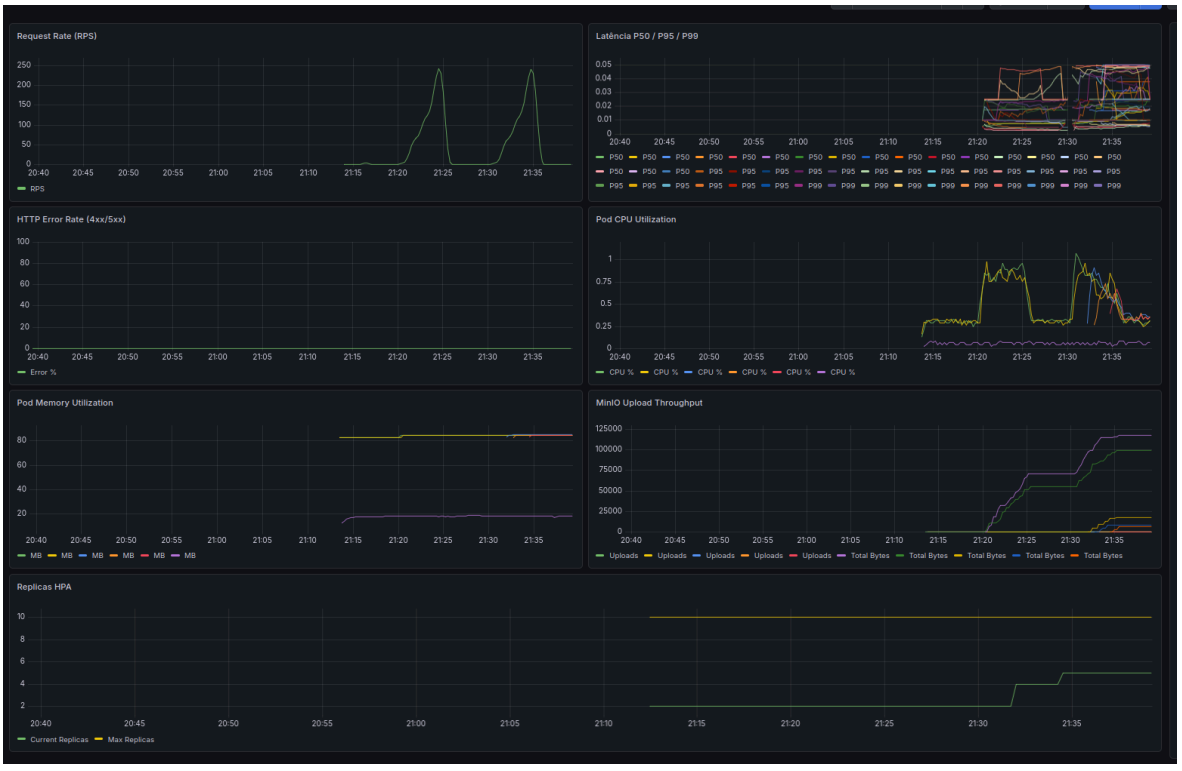


Figure 6 : Grafana dashboard - Test 2.

```
NAME                        REFERENCE                          TARGETS       MINPODS
   MAXPODS   REPLICAS   AGE
project-dropbox-api-hpa   Deployment/project-dropbox-api   cpu: 1%/60%   2
   10        2          79m
```

Figure 7 : HPA status - Test 2, before test.

```
NAME                        REFERENCE                          TARGETS      MINPODS
  MAXPODS   REPLICAS   AGE
project-dropbox-api-hpa   Deployment/project-dropbox-api   cpu: 2%/2%   2
  10        5          24m
```

Figure 8 : HPA status - Test 2, during scale-up.

```
Every 2,0s: kubectl get...  luis-IdeaPad-Flex-5-14ABR8: Sat May 30 21:41:10 2026

NAME                        REFERENCE                          TARGETS      MINPODS
  MAXPODS   REPLICAS   AGE
project-dropbox-api-hpa   Deployment/project-dropbox-api   cpu: 1%/2%   2
  10        4          29m
```

*Figure 9 : HPA status - Test 2, after cooldown. *

```
time="2026-05-30T21:42:10Z" level=info msg="\n=== LEI DE LITTLE ===" source=cons
ole
time="2026-05-30T21:42:10Z" level=info msg="Latência média:             3.07ms"
source=console
time="2026-05-30T21:42:10Z" level=info msg="P95 Latência:               18.70ms"
 source=console
time="2026-05-30T21:42:10Z" level=info msg="Throughput (RPS):           11.86" s
ource=console
time="2026-05-30T21:42:10Z" level=info msg="Erro Rate:                  19.03%"
source=console
time="2026-05-30T21:42:10Z" level=info msg="Concorrência estimada (N):  0.04 req
uests" source=console

running (5m01.0s), 00/50 VUs, 3415 complete and 0 interrupted iterations
default ✓ [ 100% ] 00/50 VUs  5m0s
```

Figure 10 : k6 load test output - Test 3 (HPA target: 2% patch, sleep: random 1–3s).

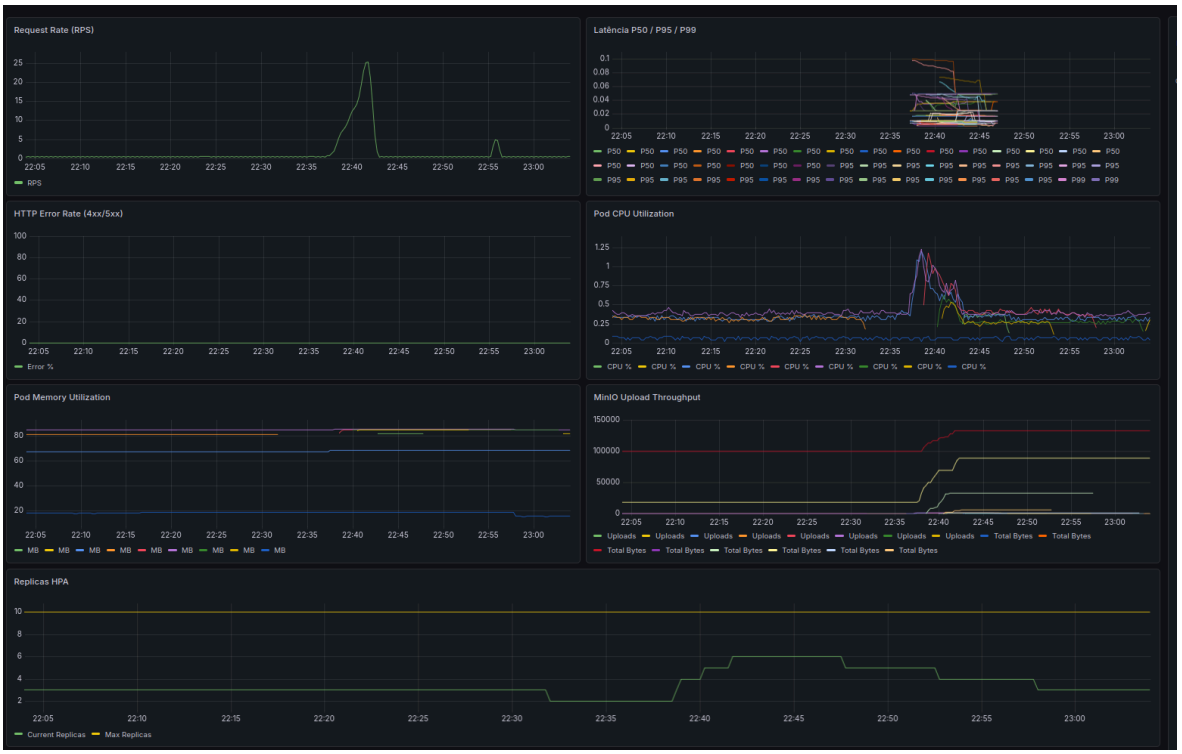


Figure 11 : Grafana dashboard - Test 3.

```
NAME                          REFERENCE                          TARGETS       MINPODS
  MAXPODS   REPLICAS   AGE
project-dropbox-api-hpa   Deployment/project-dropbox-api   cpu: 1%/2%   2
  10        6          91m
```

Figure 12 : HPA status - Test 3, peak load.